Project Report

Script-Controlled ACL - Restrict Record Access Based on Field Value

Team ID: SWTID-2026-8643

Platform: ServiceNow

Table of Contents

1. Introduction
2. Ideation Phase
3. Requirement Analysis
4. Project Design
5. Project Planning & Scheduling
6. Functional and Performance Testing
7. Results
8. Advantages & Disadvantages
9. Conclusion
10. Future Scope
11. Appendix

1. INTRODUCTION

1.1 Project Overview

This project uses script-controlled Access Control Lists (ACLs) in ServiceNow to manage who can view, create, edit and delete records in the custom Institution Details table (u_institution_details). Users receive custom roles (bb1 to bb4), while the READ ACL combines the role requirement, a Branch condition (EEE) and an advanced script. The bb1 role allows EEE records only, users without the role see no records, and administrators retain full access.

1.2 Purpose

2. IDEATION PHASE

2.1 Problem Statement

2.2 Empathy Map Canvas

Persona: an EEE branch coordinator who works with student records. The map below is a draft for the team to refine.

Says	Thinks
"I should only see EEE students." "Who can delete these records?"	Other branches' data is not my business. A wrong edit or deletion would be hard to undo.
Does	Feels
Opens the Institution Details list, searches students, updates descriptions.	Confident when only relevant records appear; concerned when sensitive data is visible to everyone.

2.3 Brainstorming

Options the team considered for restricting access:

Idea	Outcome
Role-only ACL on the table	Simple, but cannot limit records by Branch
ACL with a data condition (Branch is EEE)	Limits records by field value; selected
Script-controlled ACL (advanced)	Allows custom logic such as the admin check; selected
Separate ACL per operation (read, create, write, delete)	Gives fine control with roles bb1 to bb4; selected
Separate tables per branch	Rejected: duplicates data and is harder to maintain

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User action	Experience / outcome
Sign in	EEE User signs in (or is impersonated)	Access depends on the assigned roles

View records	Opens u_institution_details.list	Sees only EEE records (bb1)
Add a record	Clicks New	Button shown only with bb2
Stage	**User action**	**Experience / outcome**
Edit a record	Edits Description in the list	Allowed only with bb3
Delete a record	Actions > Delete, confirms	Allowed only with bb4

3.2 Solution Requirement

ID	Requirement	Type
FR-1	Create a user and custom roles bb1, bb2, bb3, bb4	Functional
FR-2	Create table u_institution_details with Branch, Email, Phone Number and other fields	Functional
FR-3	READ ACL: show only EEE records to bb1 users; admins see all	Functional
FR-4	CREATE, WRITE and DELETE ACLs tied to bb2, bb3 and bb4	Functional
NFR-1	Unauthorized users must see no records	Security
NFR-2	ACL evaluation should not noticeably slow list loading	Performance

3.3 Data Flow Diagram

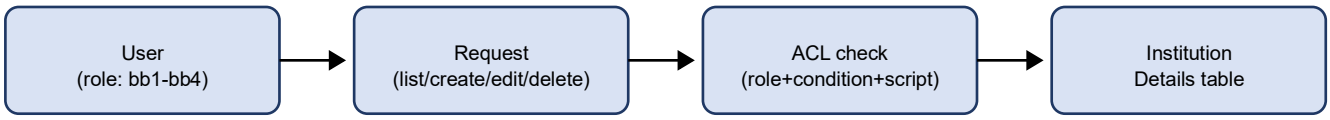


Figure 1: Data flow of an access request

3.4 Technology Stack

Component	Technology
Platform	ServiceNow instance (Global application scope)
Data	Custom table u_institution_details (fields: auto number, reference, choice, string)
Security	Roles, record ACLs (read, create, write, delete), data conditions
Scripting	Server-side JavaScript (ECMAScript 2021 mode) using gs.hasRole()
Testing	User impersonation and elevated security_admin role

4. PROJECT DESIGN

4.1 Problem Solution Fit

Problem	Solution
All branch records visible to every user	READ ACL with Branch is EEE and role bb1
Anyone could add, edit or delete	Separate ACLs requiring bb2, bb3, bb4

Admins must not be locked out	Admin check in the script and Admin overrides

4.2 Proposed Solution

ACL operation	Required role	Condition	Effect
Read	bb1	Branch is EEE + script	Only EEE records visible; admin sees all
Create	bb2	None	New button available
Write	bb3	None	Fields can be edited
Delete	bb4	None	Records can be deleted

4.3 Solution Architecture

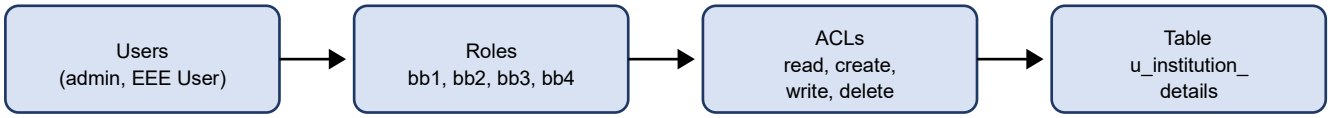


Figure 2: Solution architecture

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The lab is planned for 3 hours. The split below is a suggested schedule; adjust it to your actual timeline.

Milestone	Activity	Suggested time
1	Create user and roles bb1 to bb4	20 min
2	Create table, fields and sample records	30 min
3	Create and verify READ ACL	40 min
4	Create and verify CREATE ACL	15 min
5	Create and verify WRITE ACL	15 min
6	Create and verify DELETE ACL	15 min
7	Testing and verification	25 min
8	Documentation	20 min

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Test ID	Scenario	Expected result	Result
FT-01	Text input validation (Email, Branch, user references)	Valid inputs accepted	Pass
FT-02	Number input (auto Roll Number, Phone Number)	Roll Number auto-generates	Pass
FT-03	Access control by role (READ, CREATE, WRITE, DELETE)	Each role gets only its permitted actions	Pass

FT-04	Instance / ACL connection check	ACLs save and take effect	Pass
PT-01	List load time	Under 3 seconds	Pending
PT-02	Concurrent access	No slowdown	Pending
PT-03	Bulk data load	Loads without errors	Pending

Performance tests were not measured in the lab and remain pending. Overall validation accuracy of the role-based tests was 100%.

7. RESULTS

7.1 Output Screenshots

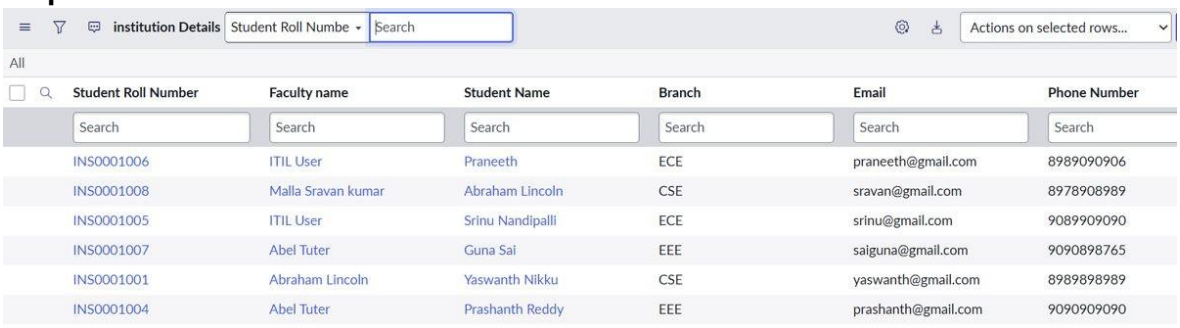


Figure 3: Institution Details records (admin view)

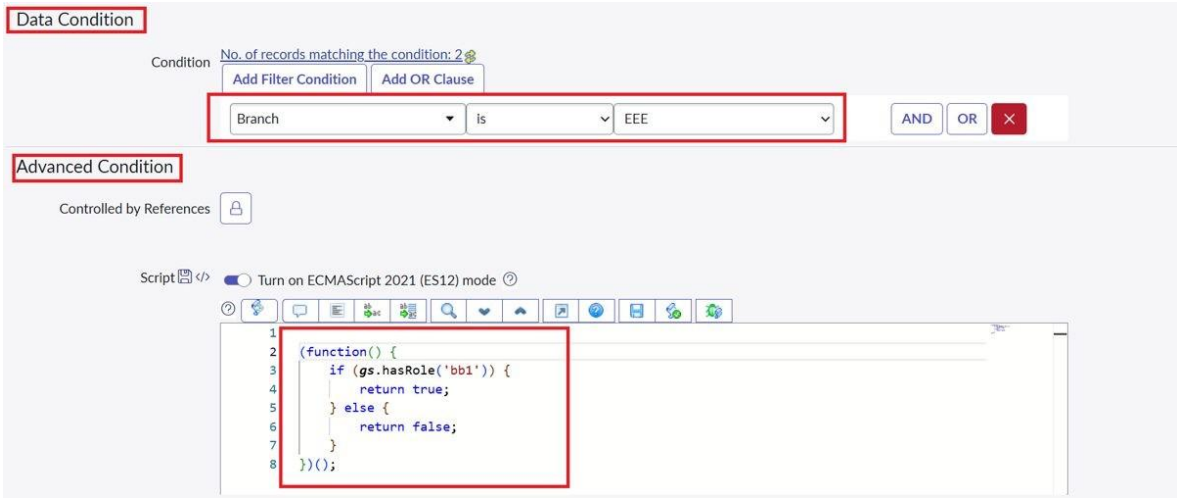


Figure 4: READ ACL data condition (Branch is EEE) and script

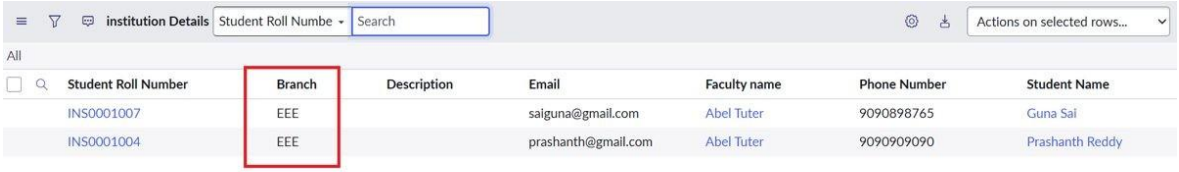


Figure 5: User with bb1 role sees only EEE records

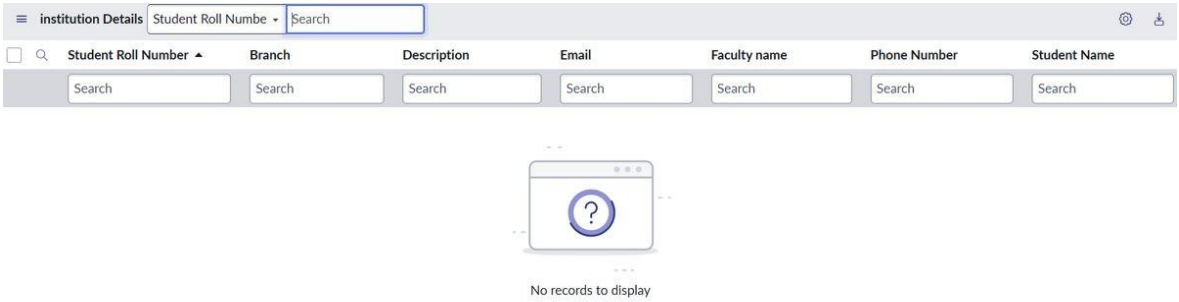


Figure 6: User without the role sees no records

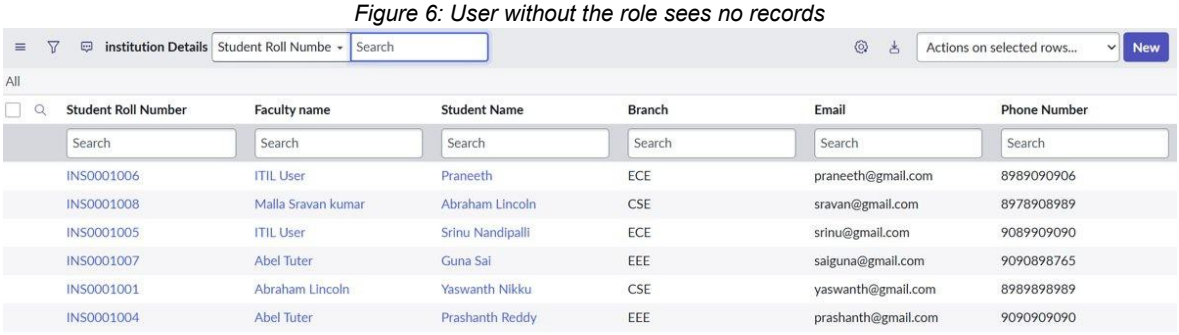


Figure 7: Admin sees all records

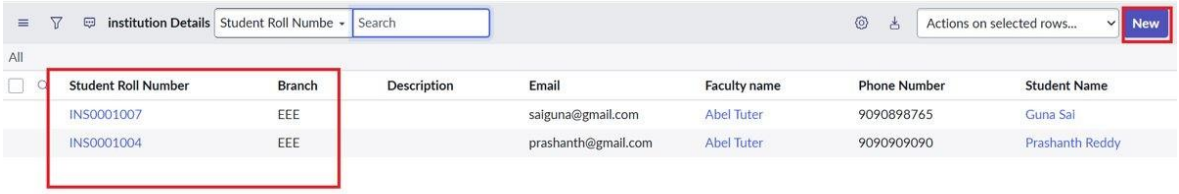


Figure 8: bb1 and bb2 users see EEE records with the New button



Figure 9: bb3 user edits the Description field



Figure 10: bb4 user deletes a record (confirmation)

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages

Record-level security based on role and field value	Roles and ACLs must be maintained as branches grow
Separate control over read, create, write and delete	Script-based ACLs need care to avoid errors
Admins keep full access	Branch-specific logic is hard-coded to EEE and bb1
No data duplication across tables	Each additional branch needs new roles or conditions

9. CONCLUSION

The project shows that script-controlled ACLs can enforce record-level security in ServiceNow. Users with bb1 see only EEE records, users without the role see nothing, and admins see everything, while bb2, bb3 and bb4 independently control creating, editing and deleting. The functional tests passed; the performance tests remain to be measured.

10. FUTURE SCOPE

- Use field-level ACLs to hide sensitive fields such as Email and Phone Number.
- Replace the hard-coded role check with a group or branch-based lookup.
- Extend the same ACL pattern to Service Portal forms and lists.
- Build a dashboard of records by branch.

11. APPENDIX

Source Code

READ ACL script (from the lab instructions):

```
(function () {   // Allow admin users full access
if (gs.hasRole('admin')) { return true; }   //
Allow only EEE branch users to see EEE records
if (gs.hasRole('bb1')) { return true; }   // Deny
access for all others   return false; })();
```

Dataset Link

Not applicable. Sample records were entered manually in the Institution Details table.

GitHub & Project Demo Link

[Add your GitHub repository link]
[Add your project demo video link]